

MANUAL DE USO

Chatflow

Editor visual de flujos de troubleshooting de alarmas PEGSA. Guía completa de tipos de nodo, reglas de modelado, validaciones y despliegue.

Documento técnico · Versión 18 de junio de 2026

Elaborado por Innovaitors S.A.S. para PEGSA — Plantas de Energía S.A.

Editor visual de flujos de troubleshooting de alarmas PEGSA

Chatflow es el editor con el que se construyen los flujos de troubleshooting de las alarmas PEGSA. En lugar de escribir el YAML a mano, el flujo se arma sobre un lienzo con nodos y conexiones, se prueba con el simulador de chat integrado y se exporta como un `.zip` que el chatbot consume en producción.

Este manual describe cada tipo de nodo, los campos que se editan en cada uno y las reglas de modelado que debe cumplir un flujo para interpretarse correctamente en el chatbot.

Contenido

1. Flujo de trabajo
 2. Tipos de nodo
 3. Reglas de conexión
 4. Reglas de modelado
 5. Validaciones automáticas
 6. Acciones del nodo de condición
 7. Traducción a YAML
 8. Exportar, importar y desplegar
 9. Simulador de chat
 10. Recomendaciones de uso
-

1. Flujo de trabajo

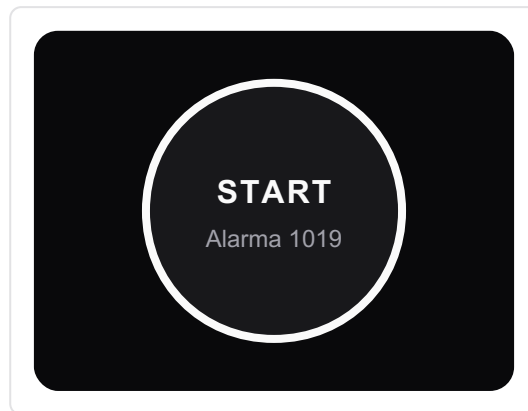
El ciclo de trabajo para una alarma tiene cinco pasos:

1. Diseñar el flujo en el lienzo (nodos y conexiones).
2. Validar: el editor marca con un signo  amarillo los nodos mal configurados.
3. Probar el recorrido con el simulador de chat.
4. Exportar el archivo de la alarma (`alarma<CÓDIGO>.zip`).
5. Desplegar ese `.zip` con el script `deploy_alarmas.sh` , que lo sube a S3 y actualiza el índice del chatbot.

Todo camino del flujo debe terminar en un nodo final —un mensaje de cierre con la acción *Create Ticket* — seguido de un nodo *Exit*.

2. Tipos de nodo

2.1 Inicio (Start)



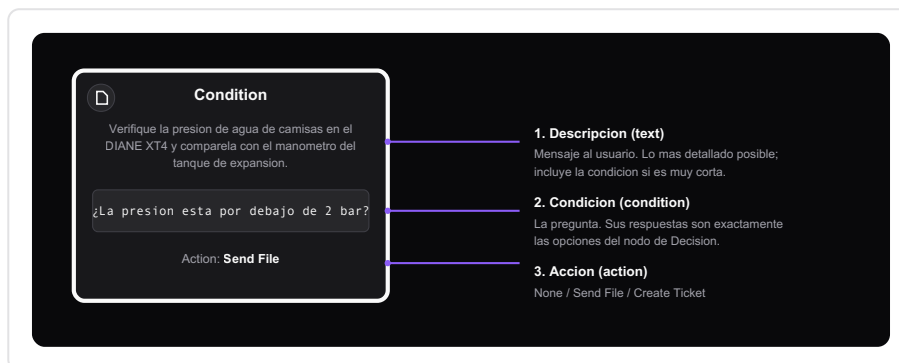
Marca el comienzo del flujo de una alarma. Hay uno solo por flujo y no se puede borrar.

Campos:

- **Código de alarma** (`alarmCode`): el código de cuatro dígitos, por ejemplo `1019`.
- **Tipo de alarma** (`alarmType`): `warning` por defecto.

El código de Start define el `id` y la `description` del grafo en el YAML (`graph_alarm_1019`).

2.2 Condición (Condition)



Es el nodo de trabajo del flujo: muestra un mensaje al usuario y, por lo general, le hace una pregunta. Tiene tres partes, editables con doble clic sobre el nodo:

Campo	Qué es	Cómo llenarlo
Descripción (<code>text</code>)	El mensaje que ve el usuario	Lo más detallado posible: instrucción clara, valores, ubicaciones y contactos.
Condición (<code>condition</code>)	La pregunta que se le hace al usuario	Una pregunta cuyas respuestas son exactamente las opciones del nodo de Decisión que sigue.
Acción (<code>action</code>)		

Campo	Qué es	Cómo llenarlo
	Acción opcional al llegar al nodo	<code>None</code> , <code>Send File</code> (adjuntar PDF o imagen) o <code>Create Ticket</code> (crear ticket).

Un nodo de Condición siempre precede a un nodo de Decisión, Exit o Go To, y tiene exactamente una conexión de salida.

2.3 Decisión (*Decision*)

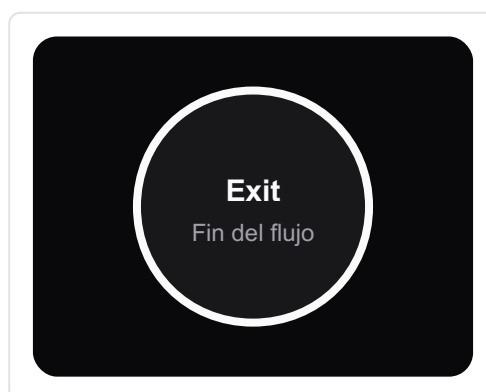


Contiene las respuestas a la pregunta planteada en el nodo de Condición anterior.

- Trae dos opciones por defecto: Sí y No.
- También admite opciones numéricas (`1` , `2` , `3` ...) para menús, u otros valores.
- Las opciones deben ser cortas y distintas entre sí.

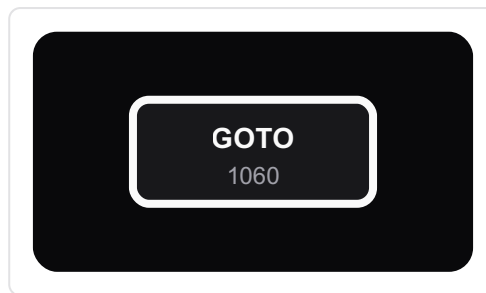
Cada opción de la Decisión se conecta con una rama del flujo. La etiqueta de cada conexión que sale de la Decisión debe coincidir con una de sus opciones.

2.4 Salida (*Exit*)



Marca el fin de un camino. Va después del nodo de Condición que entrega el mensaje de cierre. En el YAML se traduce como `next: "end"` .

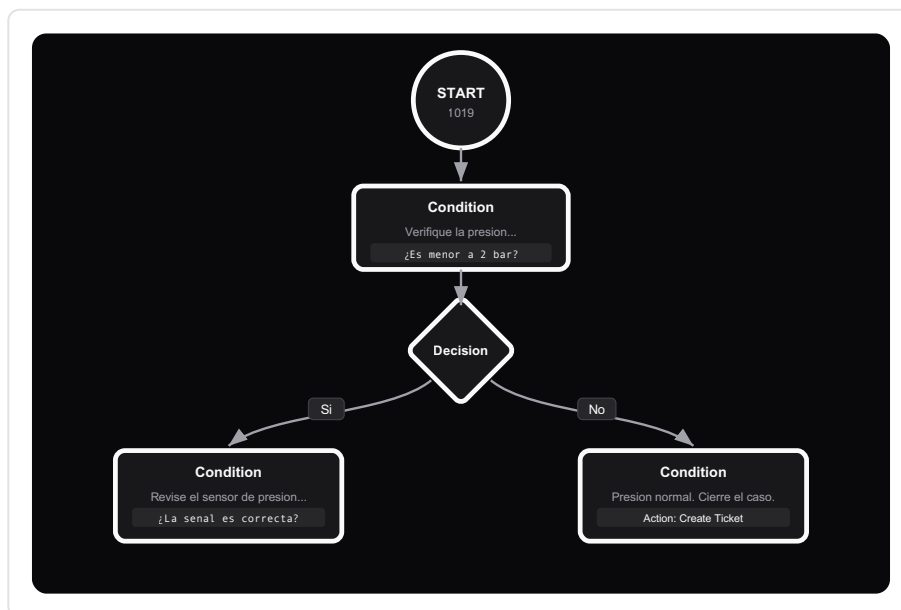
2.5 Salto (Go To)



Salta a otra alarma: indica qué flujo debe abrir el chatbot a continuación.

- Campo a llenar: el código de cuatro dígitos de la alarma destino, por ejemplo **1060**.
- Se usa cuando el troubleshooting continúa en otra alarma.

3. Reglas de conexión



El patrón base se repite a lo largo del flujo:

Start → Condition → Decision → (una rama por opción) → Condition → Decision → ...

- Start conecta con la primera Condición.
- Cada Condición tiene una sola salida, hacia una Decisión, un Exit o un Go To.
- Cada Decisión abre una rama por opción, y cada rama lleva normalmente a una nueva Condición.

4. Reglas de modelado

Un flujo válido —que el chatbot interpreta correctamente— cumple estas reglas:

1. **Cada rama de una Decisión va a un nodo de Condición.** Las opciones (Sí/No, 1/2/3...) salen de la Decisión hacia condiciones que continúan el diagnóstico, o hacia un mensaje final seguido de Exit.
2. **El campo Condición contiene la pregunta cuyas respuestas son las opciones de la Decisión que sigue.** Si la Decisión es Sí/No, la condición es una pregunta de sí o no ("¿La presión está por debajo de 2 bar?"). Si la Decisión es numérica, la condición enumera o pregunta por esas opciones.
3. **La Descripción es el texto que el chatbot muestra en ese paso.** Puede contener instrucciones, valores, ubicaciones y contactos. Es opcional: en pasos donde la pregunta de la Decisión (Sí/No o numérica) ya es suficiente, la descripción puede quedar vacía.
4. **Todos los caminos se cierran.** Cada rama termina en un mensaje final con la acción *Create Ticket* seguido de un nodo Exit, o en un Go To si el caso continúa en otra alarma.
5. **Las acciones se usan según su propósito:** *Send File* para adjuntar el manual o diagrama que apoya ese paso, *Create Ticket* en los mensajes finales.

5. Validaciones automáticas

El editor señala con un  amarillo los nodos con problemas. Los avisos y su resolución:

Nodo de Condición

- "Condition node is not connected to any other node." — conéctalo a una Decisión, Exit o Go To.
- "Condition node has more than one outgoing connection." — debe tener una sola salida.
- "Condition node must be connected to a Decision, Exit or Go To node." — no conectes una condición directamente a otra condición.

Nodo de Decisión

- "Not all options are connected." — falta conectar alguna opción.
- "An outgoing connection's label does not match any defined option." — la etiqueta de la conexión no coincide con ninguna opción; corrige el texto.
- "There are duplicate option connections." — hay dos conexiones con la misma etiqueta.

Nodo Go To

- "Go To node must have a 4-digit alarm code." — escribe un código de alarma de cuatro dígitos.

6. Acciones del nodo de condición

Acción	YAML generado	Cuándo usarla
None	(sin <code>action</code>)	Paso normal del flujo.
Send File	<code>action: Enviar archivo</code> <code><archivo></code>	Adjuntar un PDF, imagen o video de apoyo (manual, diagrama, ubicación). El archivo se incluye en <code>extra_metadata/</code> .
Create Ticket	<code>action:</code> <code>"create_ticket_in_db"</code>	En los mensajes finales, para registrar el caso de soporte.

7. Traducción a YAML

Un nodo de Condición con una pregunta y una Decisión Sí/No se exporta así:

```
- id: "condition-1772816716797"
  text: "Antes de iniciar el troubleshooting, descargue y respalde los logs.
  Adjunto una instrucción técnica que le puede ayudar."
  action: Enviar archivo exportacion_de_parametros_diane_xt4_p.pdf
  decision:
    condition: "¿Confirma que descargó los logs?"
    yes: "condition-1771590122093"
    no: "condition-1772817909063"
```

Un mensaje final (cierre y ticket) se exporta así:

```
- id: "condition-1771528448343"
  text: "Comuníquese con la línea de soporte de PEGSA: 317 300 6466 ..."
  action: "create_ticket_in_db"
  next: "end"
```

Equivalencias:

- Una rama hacia Exit se traduce como `next: "end"`.
- Una rama hacia Go To se traduce como `next: goto_<CÓDIGO>`.
- Las opciones de la Decisión se convierten en las claves del bloque `decision` (`yes` / `no`, números, o la etiqueta en minúsculas con `_`).

8. Exportar, importar y desplegar

- **Exportar ZIP:** genera `<CÓDIGO>.zip` (p. ej. `1022.zip`) con el flujo (`alarma<CÓDIGO>.yaml`), las posiciones de los nodos (`graph_layout_metadata.json`) y la carpeta `extra_metadata/` con los archivos adjuntos. Esta es la versión que se despliega.

- **Importar ZIP:** vuelve a cargar un flujo existente para seguir editándolo; reconstruye nodos, posiciones y archivos.

8.1 Desplegar el ZIP con `deploy_alarmas.sh`

El ZIP que produce Chatflow es **directamente desplegable**: no hay que editarlo, descomprimirlo ni renombrar nada. El archivo `alarma<CÓDIGO>.yaml` ya incluye, al final y comentado tras la línea `# ---`, el bloque de índice (`file_name` , `alarm_type` , `extra_metadata`) que el script necesita; Chatflow lo genera automáticamente al exportar.

Requisitos del ZIP (los cumple el export de Chatflow tal cual):

- El nombre debe ser el **código numérico** de la alarma: `<CÓDIGO>.zip` (p. ej. `1022.zip`). El script rechaza nombres que no sean numéricos.
- Contiene `alarma<CÓDIGO>.yaml` con el bloque `# ---` al final.
- Carpeta `extra_metadata/` con los adjuntos referenciados por las acciones *Send File*.

Uso del script:

```
deploy-alarmas 1022          # una alarma por código (busca ./1022.zip)
deploy-alarmas ruta/1022.zip # una alarma por ruta
deploy-alarmas                # procesa TODOS los *.zip del directorio actual
```

Qué hace por cada ZIP (todo por SSH al servidor; los datos de alarmas viven en S3, no en el servidor):

1. Copia el ZIP al servidor y lo descomprime en un directorio temporal.
2. Extrae y descomenta el bloque de índice (`# ---`) del `alarma<CÓDIGO>.yaml`.
3. Compara `extra_metadata/` contra lo que ya hay en S3 y reporta los cambios: `+` archivo nuevo, `-` sin cambios, `x` eliminado (estaba en S3 pero ya no en el ZIP).
4. Reemplaza la carpeta `s3://pegas-chatbot/Flujos/<CÓDIGO>/` con el contenido del ZIP.
5. Inserta o reemplaza el bloque `"<CÓDIGO>":` dentro de `Alarms:` en el `data/index.yaml` del servidor, con backup previo y validación del YAML resultante (si queda inválido, restaura el backup automáticamente).
6. Cuando termina todas las alarmas, recarga el bot (`docker compose` de `whatsapp-app` y `chatbot_pegas`) para que tome los cambios.

Cada corrida deja un resumen de los cambios en el log local `deploy_alarmas.log`.

En resumen: exportas el ZIP en Chatflow → `deploy-alarmas <código>` → el bot queda actualizado. No se editan a mano los archivos de alarmas del servidor; el chatbot siempre lee desde S3.

9. Simulador de chat

El simulador integrado recorre el flujo respondiendo como lo haría el usuario. El editor resalta el camino recorrido, lo que permite verificar que todas las ramas lleguen a un final correcto antes de exportar.

10. Recomendaciones de uso

Pautas para aprovechar las funciones del editor:

- Redacta las condiciones como preguntas, de modo que coincidan con las opciones de la Decisión que sigue (Sí/No o numéricas).
- Usa opciones de Decisión cortas y distintas entre sí: Sí/No para preguntas binarias y numéricas para menús.
- Adjunta con *Send File* el manual, diagrama o instrucción técnica que apoya ese paso.
- Cierra cada camino con un mensaje final y, cuando corresponda, *Create Ticket* y un nodo Exit.
- Usa el simulador de chat para recorrer el flujo y verificar que todas las ramas lleguen a un final antes de exportar.